

# Fonctionnement des GNNs

Sherelle Kana

March 21, 2024

# Sommaire

- 1 **Introduction**
- 2 **Rappel**
- 3 **Fonctionnement**
- 4 **Avantages**
- 5 **Limites**
- 6 **Notions importantes**

# Introduction

---

- Un GNN est un réseau de neurones adapté pour la manipulation des données structurées sous forme de graphes.
- Il a pour objectif d'apprendre la structure d'un graphe enfin d'effectuer une tâche ( classification , régression, etc..)

## Rappel

---

Un graphe est un ensemble de nœuds et de liens. Formellement, un graphe est le couple  $G = (V, E)$ , où  $V = \{1, \dots, n\}$  est l'ensemble des  $n$  nœuds, et  $E = \{(i, j) | i, j \in V\}$  est l'ensemble des liens entre eux.

Un graphe peut également être représenté par une matrice d'adjacence  $A$  de taille  $n \times n$  telle que:

$$A_{ij} = \begin{cases} 1 & \text{si } (i, j) \in E, \\ 0 & \text{sinon.} \end{cases}$$

Pour une tâche donnée on peut considérer soit:

- Les caractéristiques des nœuds
- Les caractéristiques des liens
- Les caractéristiques du graphe

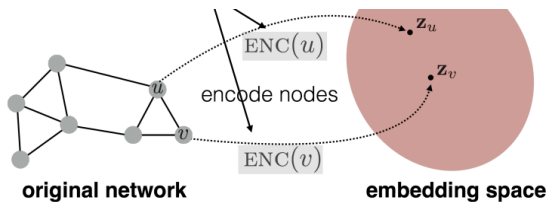
## NB

Dans la suite nous considérons que pour notre tâche, on se focalise sur les nœuds. Il est donc question de produire des embeddings des nœuds de notre graphe.

# Fonctionnement

---





On veut quitter du graphe original a un espace de représentation qui est similaire à l'espace original. L'approche naïve consiste à vérifier cette similitude en utilisant le produit scalaire. Cette approche est limitée par le fait que les embeddings des nœuds sont indépendants les uns les autres et traités de façon individuelle.

Graphe



GNN



Embeddings des noeuds

### Note

Une couche GNN décrit donc un mécanisme plus complexe pour représenter chaque nœud dans un espace embedding.

Pour expliquer ce mécanisme , considérons pour notre graphe en entrée :

- À chaque nœud  $i$  on associe un vecteur  $x_i$  de taille  $d$  contenant les caractéristiques du nœud.
- La matrice  $X$  de taille  $(n,d)$  contenant les  $x_i$  des  $n$  nœuds du graphe
- La matrice  $A$ , matrice d'adjacence de taille  $(n,n)$
- $N_i$  le voisinage du noeud  $i$  est tel que  $N_i = \{j \mid A_{ij} = 1\}$

Un GNN est une fonction  $\mathbf{f}$  qui:

- Prend en entrée les matrices  $\mathbf{X}$  et  $\mathbf{A}$
- Renvoie une matrice  $\mathbf{H}$  d'embeddings des nœuds du graphe  
 $\mathbf{H} = \mathbf{f}(\mathbf{X}, \mathbf{A})$

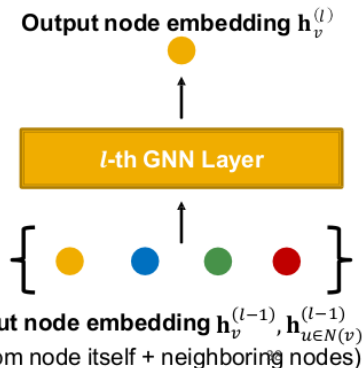
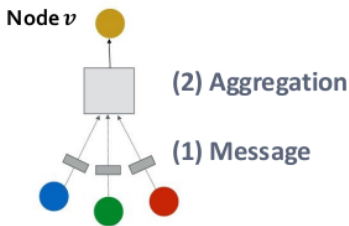
$\mathbf{f}(\mathbf{X}, \mathbf{A})$  est invariant à la permutation. Soit  $\mathbf{P}$  une matrice de permutation  $\mathbf{X}' = \mathbf{P}\mathbf{X}$ ,  $\mathbf{A}' = \mathbf{P}\mathbf{A}\mathbf{P}^T$   $\mathbf{f}(\mathbf{X}', \mathbf{A}') = \mathbf{P}\mathbf{f}(\mathbf{X}, \mathbf{A})$  cette propriété garantit que la prédiction dans la couche GNN soit indépendant de l'ordre des nœuds.

### Note

Cette propriété découle du fait que les graphes soient invariants à la permutation des nœuds, contrairement aux images.

La couche GNN est formée de deux processus:

- (1) Message
- (2) Agrégation



Ces deux processus sont appliqués à un nœud  $i$  de la matrice  $X$  et à ses nœuds voisins  $N_i$  obtenus grâce à la matrice  $A$ . La couche produit un embeddings du nœud  $i$  à cette couche.

En résumé on peut schématiser le mécanisme du GNN comme suit:



### Note

On peut empiler plusieurs couche GNN.

## Processus Message

Considérons que nous sommes à la couche  $l$ . Le message est créé par chaque noeud et transmis aux autres nœuds, le message pour un nœud  $u$  à la couche  $l$  est obtenu:

$$\mathbf{m}_u^{(l)} = \text{MSG}^{(l)} \left( \mathbf{h}_u^{(l-1)} \right), u \in \{N(v) \cup v\}$$

Pour une couche linéaire  
poids

$$m_u^{(l)} = W^{(l)} h_u^{(l-1)}$$

où  $\mathbf{W}$  est le vecteurs des

## Processus Agrégation

Chaque nœud agrège les messages reçus des autres nœuds. L'agrégation est faite de la façon suivante:

$$\mathbf{h}_v^{(l)} = \text{AGG}^{(l)} \left( \left\{ \mathbf{m}_u^{(l)}, u \in N(v) \right\}, \mathbf{m}_v^{(l)} \right)$$

AGG peut être la som, moy, max



## L'entraînement dans les GNNs implique:

- L'agrégation des messages à partir des nœuds voisins
- L'application du réseau neuronal

L'application du réseau neuronal

$$\mathbf{h}_v^0 = \mathbf{x}_v$$

À la couche 0 on considère l'embedding du nœud comme étant égal au vecteur caractéristique du nœud.

L'application du réseau neuronal

$$\mathbf{h}_v^k = \sigma \left( \mathbf{W}_k \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|} + \mathbf{B}_k \mathbf{h}_v^{k-1} \right), \quad \forall k \in \{1, \dots, K\}$$

Cette formule intègre la notion de **message passing** qui consiste ou permet de faire passer le message d'un nœud vers les autres d'une couche, ceci abouti à la création d'un nouvel embedding qui sera utilisé à la couche suivante.

$\mathbf{W}_k$  c'est la matrice des poids à la couche  $k$

$\mathbf{B}_k$  c'est le vecteur des biais à la couche  $k$

$\sigma$  la fonction d'activation : ReLu

L'application du réseau neuronal

$$\mathbf{z}_v = \mathbf{h}_v^K$$

$$\mathbf{h}_v^k = \sigma \left( \mathbf{W}_k \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|} + \mathbf{B}_k \mathbf{h}_v^{k-1} \right), \quad \forall k \in \{1, \dots, K\}$$

Durant l'entraînement entre les k couches du GNN, la qualité des vecteurs embeddings à chaque couche est améliorée en appliquant à  $\mathbf{z}_v$  une fonction de perte, il est donc question d'optimiser cette fonction par la méthode descente du gradient ou une de ses variantes afin de mettre à jour les paramètres de l'agrégation  $\mathbf{W}_k$  et  $\mathbf{B}_k$  utilisés durant l'entraînement.

## Utilisation des embeddings

Une fois les embeddings générés on peut effectuer une tâche de classification supervisée, en définissant une fonction de perte telle que suit:

$$\mathcal{L} = \sum_{v \in V} y_v \log(\sigma(\mathbf{z}_v^T \boldsymbol{\theta})) + (1 - y_v) \log(1 - \sigma(\mathbf{z}_v^T \boldsymbol{\theta}))$$

Entropie croisée

très utilisée pour les tâches de classification.

## Note

Les processus message et agrégation seront modifiés selon le type de GNN.

## Advantages

---

- Interprétabilité: grâce à l'analyse des embeddings de nœuds et des messages propagés entre les nœuds.
- Flexibilité: Les GNNs peuvent traiter des données structurées sous forme de graphes, ce qui les rend adaptables à une grande variété d'applications.

# Limits

---

- Deux nœuds ayant la même matrice d'adjacence auront la même matrice d'embedding en sortie. Par conséquent, les couches peuvent faire oublier au réseau les informations spécifiques aux nœuds si nous prenons simplement une moyenne sur tous les messages.



## Notions importantes

---

## Champs réceptif

Celui ci représente l'ensemble des nœuds qui contribuent a la formation de l'embedding d'un nœud donné.

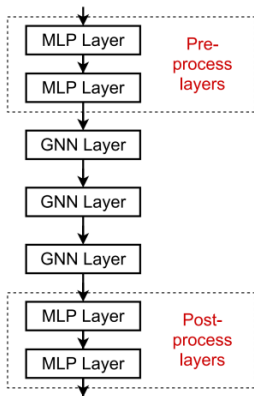
Plus on empile les couches plus ce champs grandit et le nombre de voisins du nœud considéré devient grand.

Ce qui conduit à la production des embeddings de plus en plus similaires d'une couche à une autre.

Pour éviter ce problème, on peut améliorer l'expressivité du GNN

## Expressivité

Elle désigne la capacité du GNN à distinguer les nœuds. Elle dépend de la manière dont sont agrégés les nœuds.



Pour cela on peut ajouter un perceptron multi couche (MLPs) à l'entrée et sortie du GNN. Celui-ci va prendre en entrée la **matrice  $X$  des caractéristiques des nœuds** et produire en sortie le vecteur d'embeddings.

## Influences des MLP

- MLP peut apprendre des interactions complexes entre les caractéristiques des nœuds ce qui permet d'obtenir des embeddings plus discriminants.
- MLP remplace l'agrégation linéaire standard ce qui permet d'apprendre des transformations non linéaires sur les embeddings des nœuds ce qui enrichit la capacité de modélisation des GNNs.